Experiment: Implement K-Nearest Neighbors (KNN) and Evaluate Model Performance

1. Dataset Source

- Dataset Name: Adult Census Income Dataset
- Source: Hugging Face
- Link: https://huggingface.co/datasets/scikit-learn/adult-census-income

2. Dataset Description

The Adult Census Income dataset is used to predict whether an individual earns more than $50K per year based on demographic and employment-related attributes.

- **Number of Instances:** ~48,842
- **Number of Features:** 14 input features + 1 target variable

Features:

- age – Age of the individual
- workclass – Type of employment
- fnlwgt – Final weight assigned to individuals
- education – Education level
- education-num – Numeric representation of education
- marital-status – Marital status
- occupation – Type of occupation
- relationship – Family relationship
- race – Race category
- sex – Gender
- capital-gain – Capital gains
- capital-loss – Capital losses
- hours-per-week – Working hours per week
- native-country – Country of origin

Target Variable:

- income
  - 0 → Income ≤ 50K
  - 1 → Income > 50K

Dataset Characteristics:

- Contains both **categorical and numerical data**
- Requires **encoding of categorical features**
- Requires **feature scaling for distance-based algorithms like KNN**
- Slight class imbalance present

3. Mathematical Formulation of the Algorithm

K-Nearest Neighbors (KNN) is a **non-parametric and instance-based learning algorithm**.

Distance Calculation (Euclidean Distance):

$$d(x, x_i) = \sqrt{\sum_{j=1}^{n} (x_j - x_{ij})^2}$$

Where:

- (x) = Test data point
- (x_i) = Training data point
- (n) = Number of features

Classification Rule:

$$\hat{y} = \text{mode}(y_1, y_2, ..., y_k)$$

- The algorithm finds the **K nearest neighbors**
- Assigns the class based on **majority voting**

4. Algorithm Limitations

1. Computationally expensive for large datasets
2. Requires significant memory (stores entire dataset)
3. Sensitive to feature scaling
4. Performance decreases in high-dimensional datasets (curse of dimensionality)
5. Sensitive to noise and irrelevant features
6. Selection of optimal K value is critical

5. Methodology / Workflow

Steps Involved

1. Import required libraries
2. Load dataset from Hugging Face (Adult Census Income dataset)
3. Perform data preprocessing (handling missing values, encoding categorical features)
4. Split dataset into training and testing sets
5. Apply feature scaling using StandardScaler
6. Train initial K-Nearest Neighbors (KNN) model
7. Define hyperparameter grid (values of K, distance metric, weights)
8. Perform GridSearchCV for tuning
9. Retrieve best parameters
10. Train final KNN model using optimal parameters

11. Evaluate model performance using accuracy, precision, recall, and F1-score
12. Generate predictions

Screenshots:-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from datasets import load_dataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
# Load a subset first (efficient)
dataset = load_dataset("scikit-learn/adult-census-income", split="train[:5000]")

df = pd.DataFrame(dataset)

# Randomly select 2000 rows
df = df.sample(n=2000, random_state=42).reset_index(drop=True)

df.head()
```

	age	workclass	fnlwgt	education	education.num	marital.status	occupation	relationship	race	sex	capital.gain	capital.loss	hours.per.week	native.country	income
0	39	State-gov	119421	HS-grad	9	Separated	Adm-clerical	Unmarried	White	Female	0	625	35	United-States	<=50K
1	53	Self-emp-not-inc	135339	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	Asian-Pac-Islander	Male	7688	0	20	China	>50K
2	42	Private	402367	Some-college	10	Married-civ-spouse	Transport-moving	Husband	Black	Male	7688	0	45	United-States	>50K
3	37	Private	200598	Assoc-acdm	12	Married-civ-spouse	Prof-specialty	Husband	White	Male	0	1740	45	United-States	<=50K
4	44	Private	35910	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	1902	56	United-States	>50K

```
df.info()
df.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             2000 non-null   int64
 1   workclass       2000 non-null   object
 2   fnlwgt          2000 non-null   int64
 3   education       2000 non-null   object
 4   education.num   2000 non-null   int64
 5   marital.status  2000 non-null   object
 6   occupation      2000 non-null   object
 7   relationship    2000 non-null   object
 8   race            2000 non-null   object
 9   sex             2000 non-null   object
 10  capital.gain    2000 non-null   int64
 11  capital.loss    2000 non-null   int64
 12  hours.per.week  2000 non-null   int64
 13  native.country  2000 non-null   object
 14  income          2000 non-null   object
dtypes: int64(6), object(9)
memory usage: 234.5+ KB
Index(['age', 'workclass', 'fnlwgt', 'education', 'education.num',
       'marital.status', 'occupation', 'relationship', 'race', 'sex',
       'capital.gain', 'capital.loss', 'hours.per.week', 'native.country',
       'income'],
      dtype='object')
```

```
df = df.replace("?", np.nan)
df = df.dropna()
```

```
df = pd.get_dummies(df, drop_first=True)
```

```
X = df.drop('income_>50K', axis=1)
y = df['income_>50K']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

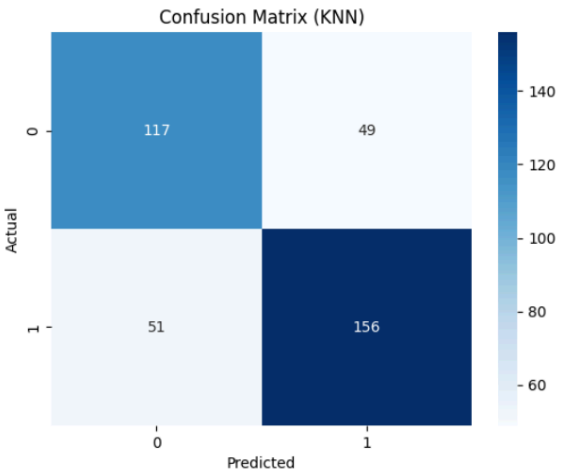
```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier()
```

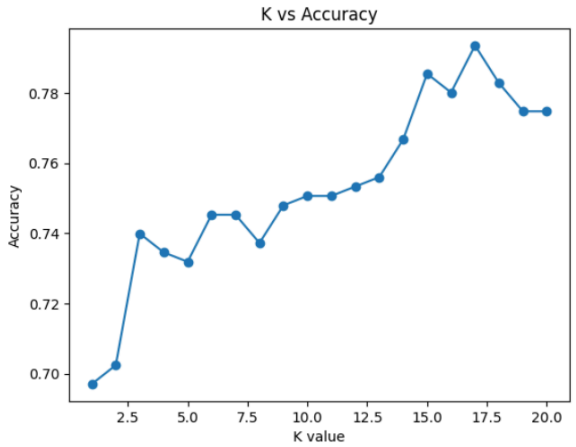
```
y_pred = knn.predict(X_test_scaled)

print("=== KNN Results ===")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
=== KNN Results ===
Accuracy: 0.7319034852546917

Classification Report:
               precision    recall  f1-score   support

       False       0.70      0.70      0.70       166
        True       0.76      0.75      0.76       207

    accuracy                           0.73       373
   macro avg       0.73      0.73      0.73       373
weighted avg       0.73      0.73      0.73       373
```

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix (KNN)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
k_values = range(1, 21)
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    y_pred_k = knn.predict(X_test_scaled)
    accuracies.append(accuracy_score(y_test, y_pred_k))

plt.plot(k_values, accuracies, marker='o')
plt.xlabel("K value")
plt.ylabel("Accuracy")
plt.title("K vs Accuracy")
plt.show()
```




```
best_k = k_values[np.argmax(accuracies)]
best_k
```

```
17
```

6. Performance Analysis

Evaluation Metrics Used:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Interpretation:

- **Accuracy:** Measures overall correctness of predictions
- **Precision:** Indicates how many predicted high-income individuals are actually high-income
- **Recall:** Measures the ability to correctly identify high-income individuals
- **F1-score:** Provides a balance between precision and recall

Observations:

- KNN performs well on structured datasets after proper preprocessing
- Feature scaling significantly improves performance
- Larger dataset size increases computation time
- Class imbalance may affect prediction of minority class

7. Hyperparameter Tuning

Parameters Tuned:

- Number of neighbors (K)
- Distance metric (Euclidean, Manhattan)
- Weight function (uniform, distance-based)

Method Used:

- GridSearchCV

Process:

- Defined a parameter grid with multiple values of K
- Used cross-validation to evaluate combinations
- Selected parameters based on highest validation accuracy

Best Parameters:

- Optimal K value selected from grid search
- Best distance metric and weight function chosen

Impact on Performance:

- Improved model accuracy and generalization
- Reduced overfitting and underfitting
- Enhanced prediction reliability

8. Conclusion

In this experiment, the K-Nearest Neighbors (KNN) algorithm was implemented to classify income levels using the Adult Census dataset. The model performed effectively after proper preprocessing and feature scaling. Hyperparameter tuning improved accuracy and overall model performance. However, KNN is computationally expensive for large datasets and sensitive to the choice of K. Overall, it is a simple and effective classification technique when properly optimized.